

8. Hyperparameter Optimization - I

Perform hyperparameter optimizations by using Grid and Random search techniques for Regression and classification problem.

A. Classification – Hyperparameter Optimization

Procedure

- Step 1: Import the required libraries and functions for classification, including GridSearchCV, RandomizedSearchCV, RepeatedStratifiedKFold, LogisticRegression, and loguniform.
- Step 2: Load the classification dataset and separate the independent variables X and dependent variable y.
- Step 3: Split the dataset into training and testing sets using train_test_split().
- Step 4: Define the classification model, such as LogisticRegression().
- Step 5: Define RepeatedStratifiedKFold for cross-validation.
- Step 6: Define the hyperparameter values for Grid Search, such as C, penalty, and solver.
- Step 7: Apply GridSearchCV() using the model, parameter grid, cross-validation method, and accuracy as the scoring measure.
- Step 8: Fit the Grid Search model using the training data.
- Step 9: Obtain the best hyperparameter combination using best_params_ and the best cross-validation score using best_score_.
- Step 10: Define the hyperparameter distributions for Random Search.
- Step 11: Apply RandomizedSearchCV() by specifying the model, parameter distributions, number of iterations, cross-validation method, and accuracy as the scoring measure.
- Step 12: Fit the Random Search model using the training data.
- Step 13: Obtain the best hyperparameter combination and best score using best_params_ and best_score_.
- Step 14: Compare the Grid Search and Random Search results and select the hyperparameter combination giving better classification performance.

B. Regression – Hyperparameter Optimization

Procedure

- Step 1: Import the required libraries and functions for regression, including GridSearchCV, RandomizedSearchCV, RepeatedKFold, Ridge, and loguniform.
- Step 2: Load the regression dataset and separate the independent variables X and dependent variable y.
- Step 3: Split the dataset into training and testing sets using train_test_split().
- Step 4: Define the regression model, such as Ridge().
- Step 5: Define RepeatedKFold for cross-validation.
- Step 6: Define the hyperparameter values for Grid Search, such as alpha and fit_intercept.
- Step 7: Apply GridSearchCV() using the model, parameter grid, cross-validation method, and neg_mean_absolute_error as the scoring measure.
- Step 8: Fit the Grid Search model using the training data.
- Step 9: Obtain the best hyperparameter combination using best_params_ and the best cross-validation score using best_score_.
- Step 10: Define the hyperparameter distributions for Random Search.
- Step 11: Apply RandomizedSearchCV() by specifying the model, parameter distributions, number of iterations, cross-validation method, and neg_mean_absolute_error as the scoring measure.
- Step 12: Fit the Random Search model using the training data.
- Step 13: Obtain the best hyperparameter combination and best score using best_params_ and best_score_.
- Step 14: Compare the Grid Search and Random Search results and select the hyperparameter combination giving better regression performance.

Solution:

```
#Classification - Grid Search and Random Search
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV,
RandomizedSearchCV, RepeatedStratifiedKFold
from sklearn.linear_model import LogisticRegression
from scipy.stats import loguniform
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
model = LogisticRegression(max_iter=5000)
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=42)
param_grid = {
    'solver': ['liblinear'],
    'l1_ratio': [0, 1],
    'C': [0.001, 0.01, 0.1, 1, 10, 100]
}
grid_search = GridSearchCV(model, param_grid=param_grid, scoring='accuracy',
cv=cv, n_jobs=-1)
grid_search.fit(X_train, y_train)
print("Grid Search")
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
param_dist = [
    {
        'solver': ['liblinear'],
        'l1_ratio': [0, 1],
        'C': loguniform(1e-5, 100)
    },
    {
        'solver': ['lbfgs'],
        'l1_ratio': [0],
        'C': loguniform(1e-5, 100)
    }
]
random_search = RandomizedSearchCV(model, param_distributions=param_dist,
n_iter=20, scoring='accuracy', cv=cv, random_state=42, n_jobs=-1)
random_search.fit(X_train, y_train)
print("Random Search")
print("Best Parameters:", random_search.best_params_)
print("Best Score:", random_search.best_score_)

#Regression - Grid Search and Random Search
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV,
RandomizedSearchCV, RepeatedKFold
from sklearn.linear_model import Ridge
from scipy.stats import loguniform
data = load_diabetes()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = Ridge()
```

```

cv = RepeatedKFold(
    n_splits=5, n_repeats=2, random_state=42
)
param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1, 10, 100],
    'fit_intercept': [True, False]
}
grid_search = GridSearchCV(
    model,
    param_grid=param_grid,
    scoring='neg_mean_absolute_error',
    cv=cv,
    n_jobs=-1
)
grid_search.fit(X_train, y_train)
print("Grid Search")
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)

param_dist = {
    'alpha': loguniform(1e-5, 100),
    'fit_intercept': [True, False]
}
random_search = RandomizedSearchCV(
    model,
    param_distributions=param_dist,
    n_iter=20,
    scoring='neg_mean_absolute_error',
    cv=cv,
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_train, y_train)
print("Random Search")
print("Best Parameters:", random_search.best_params_)
print("Best Score:", random_search.best_score_)

```

A. Classification – Grid Search and Random Search

Grid Search

Best Parameters: {'C': 100, 'l1_ratio': 1, 'solver': 'liblinear'}

Best Score: 0.9615384615384615

Random Search

Best Parameters: {'C': np.float64(57.46775499181863), 'l1_ratio': 1, 'solver': 'liblinear'}

Best Score: 0.9648351648351647

B. Regression – Grid Search and Random Search

Grid Search

Best Parameters: {'alpha': 0.001, 'fit_intercept': True}

Best Score: -44.97319035408481

Random Search

Best Parameters: {'alpha': np.float64(0.004185822729546971), 'fit_intercept': True}

Best Score: -44.98005486235784